

Portale Istituzionale del Comune di Cagliari - CMS REST API

Come usare Postman per testare le REST API del CMS

Entando Support Team

Indice

Scopo del documento	2
Versione rilascio	2
Introduzione alle Rest API del CMS	3
Export Postman	4
Environment	4
Autenticazione	6
Esempio di invocazione e note generali	10
Refresh dell'access token	11
CONTENT CMS	13
Filtri per attributo	13
Filtro per metadati	13
Concatenazione dei filtri	14
Stato pubblicazione	15
Escaping	15

Scopo del documento

Lo scopo del documento è quello di presentare come interagire con il CMS tramite REST API. Il presente documento è rivolto ai developer e ai devOps.

NOTA: anche se il focus del documento è posto sulle sole GET, la collection di Postman contiene tutte le REST per completezza, sebbene queste ultime non siano state testate sul CMS del portale istituzionale.

Versione rilascio

Versione	Data
1.0	02/12/2025

Il progetto del portale istituzionale è basato su Entando v6.5.4 - versione progetto v6.5.21.

Introduzione alle Rest API del CMS

Nel passaggio dalla versione 5.x (cloud-ready) alla versione 6.x (Cloud-native) di Entando, il supporto delle REST API è passato da Apache CFX a Spring. Questo ha provocato il rename del path degli endpoint esistenti per distinguerli con quelli nuovi. In generale, il percorso dell'endpoint del CMS è

None

www.comune.cagliari.it/portale/legacyapi/rs/<LANG>/jacms/*

Le legacy API continuano ad essere supportate su Entando 7.x, ma si consiglia di basare i nuovi sviluppi sulle REST della collection associata a questo documento.

Il payload restituito dalla piattaforma rimane JSON, ma con un formato differente a parità di contenuto informativo.

Tutti gli endpoint con una base diversa da quella riportata sopra, non sono pertinenti al CMS.

Autenticazione

Normalmente, Entando consente l'accesso ai contenuti appartenenti al gruppo free senza richiedere alcuna autenticazione. Per accedere a contenuti o asset associati a un gruppo diverso da free, oppure per eseguire operazioni di creazione, modifica o cancellazione, è invece necessario fornire un token di autorizzazione.

A partire da Entando 6, il processo di autenticazione è stato delegato a Keycloak, al fine di garantire una maggiore flessibilità e una più semplice integrazione con diversi sistemi e framework di autenticazione. Per testare le REST API del CMS è necessario,

entando

quindi, configurare il proprio client affinché utilizzi lo stesso Keycloak impiegato dall'istanza Entando.

Il protocollo di autenticazione è basato su OAuth 2.

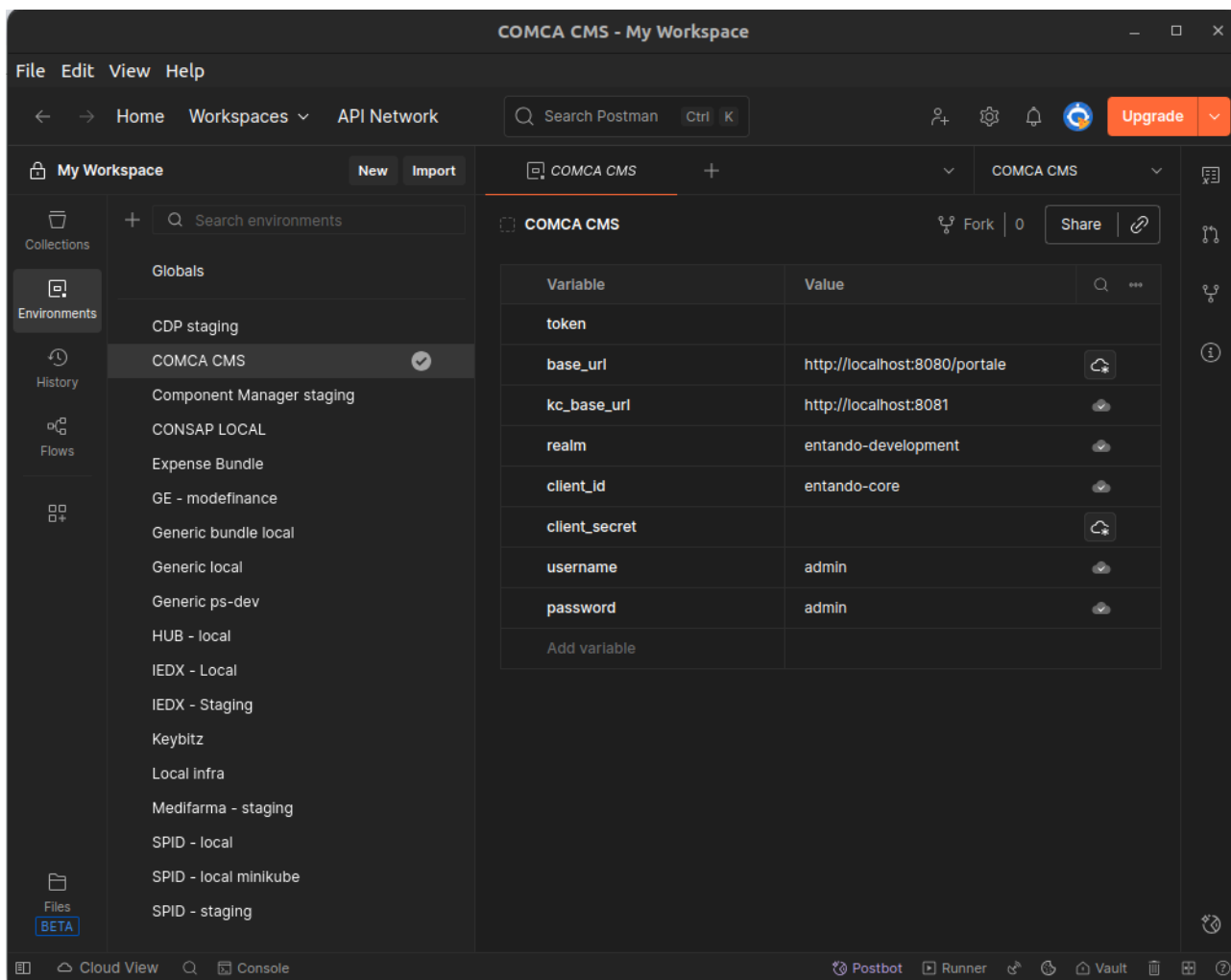
Per semplificare l'integrazione con il CMS del portale istituzionale di Cagliari, oltre al presente documento è stato messo a disposizione anche un export Postman.

Export Postman

L'export di Postman consta di due file differenti:

- un file di environment, dove il developer imposta i vari "puntamenti" a Keycloak e alla de-app di Entando
- un file con l'export vero e proprio, contenente la collection delle API del CMS.

Environment



L'environment contiene le variabili d'ambiente utilizzate nella collection REST. È necessario modificare tali variabili in base al proprio ambiente di testing.

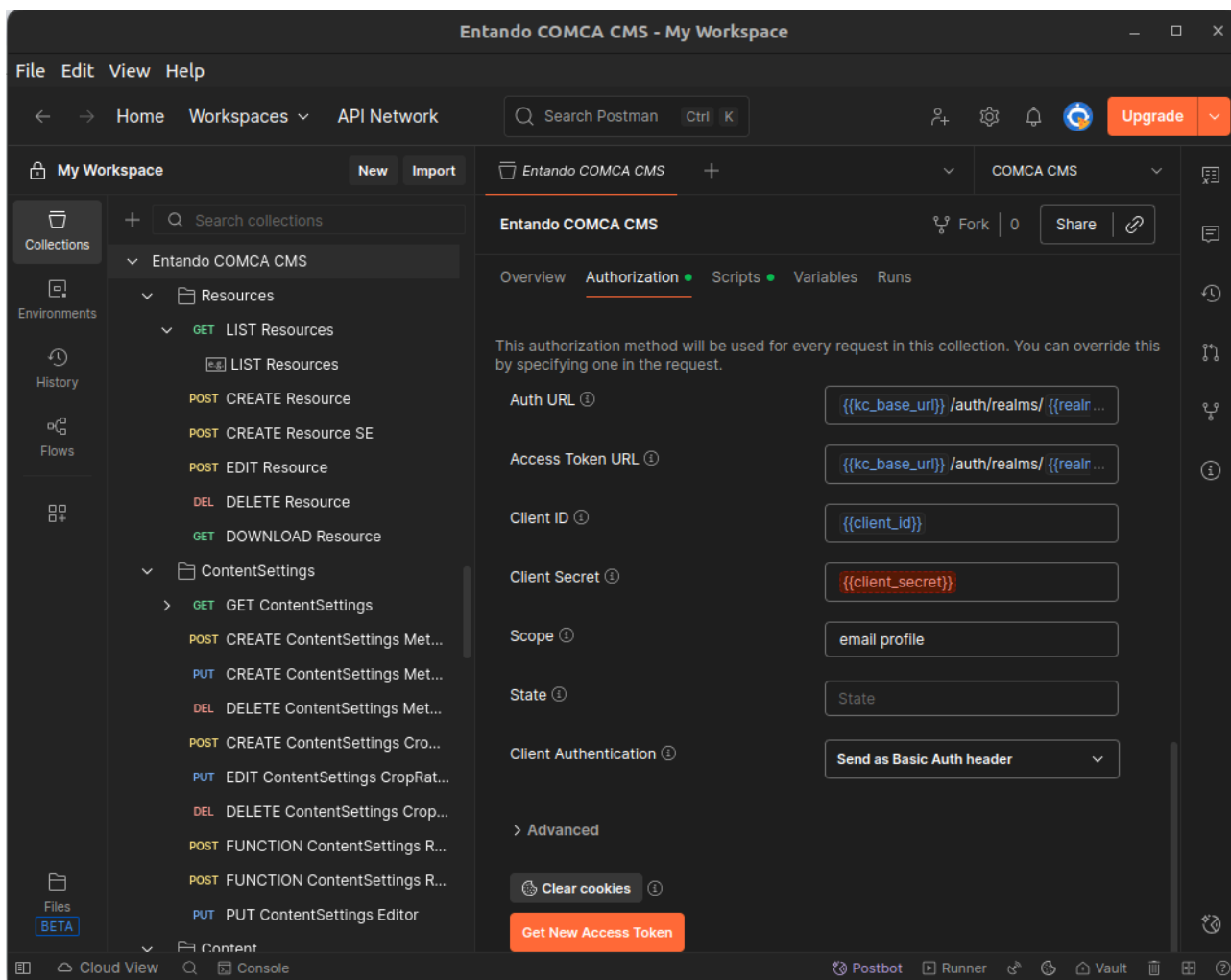
entando

Nome	Default	Descrizione
base_url	http://localhost:8080/portale	FQDN + contesto della dea-pp
kc_base_url	http://localhost:8081	Indirizzo dell'istanza di Keycloak
realm	entando-development	Realm utilizzato dalla istanza della de-app
client_id	entando-core	ID del client Keycloak utilizzato per l'autenticazione
client_secret		Secret associato al client_id
username	admin	utenza di Entando con la quale autenticarsi ¹
password	admin	password associata all'utente

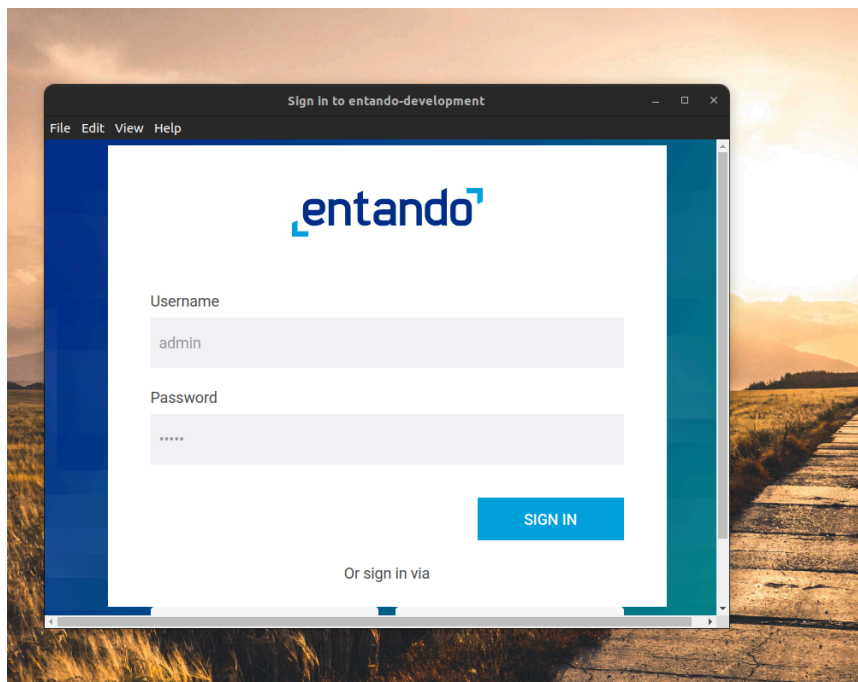
Autenticazione

Importati i file dell'environment e della collection, fare clic su “Entando COMCA CMS” -> “Authorization”; attivare l'environment “COMCA CMS” come da figura sottostante:

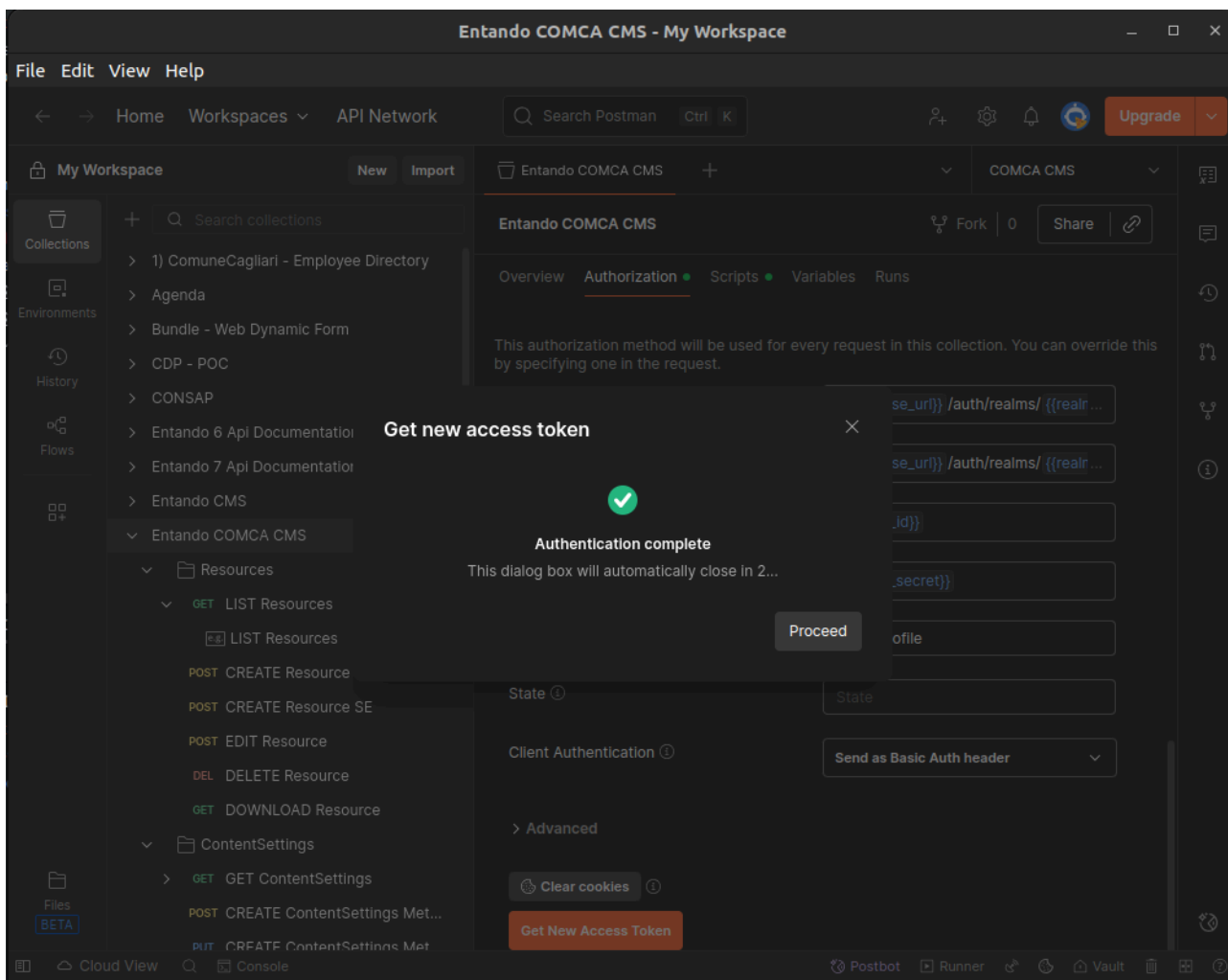
¹ Username e password servono solo per testare la REST di autenticazione (non mediata da Postman) e quindi queste variabili, nell'ambito del presente documento, possono essere ignorate.



Premere il bottone “Get new Access Token” per negoziare un nuovo access token con l'istanza di Keycloak; Postman apre una sessione di login:



Una volta inserite le credenziali, Postman mostra l'access token ottenuto:



Premere “Proceed” e quindi “use token” nella schermata successiva.

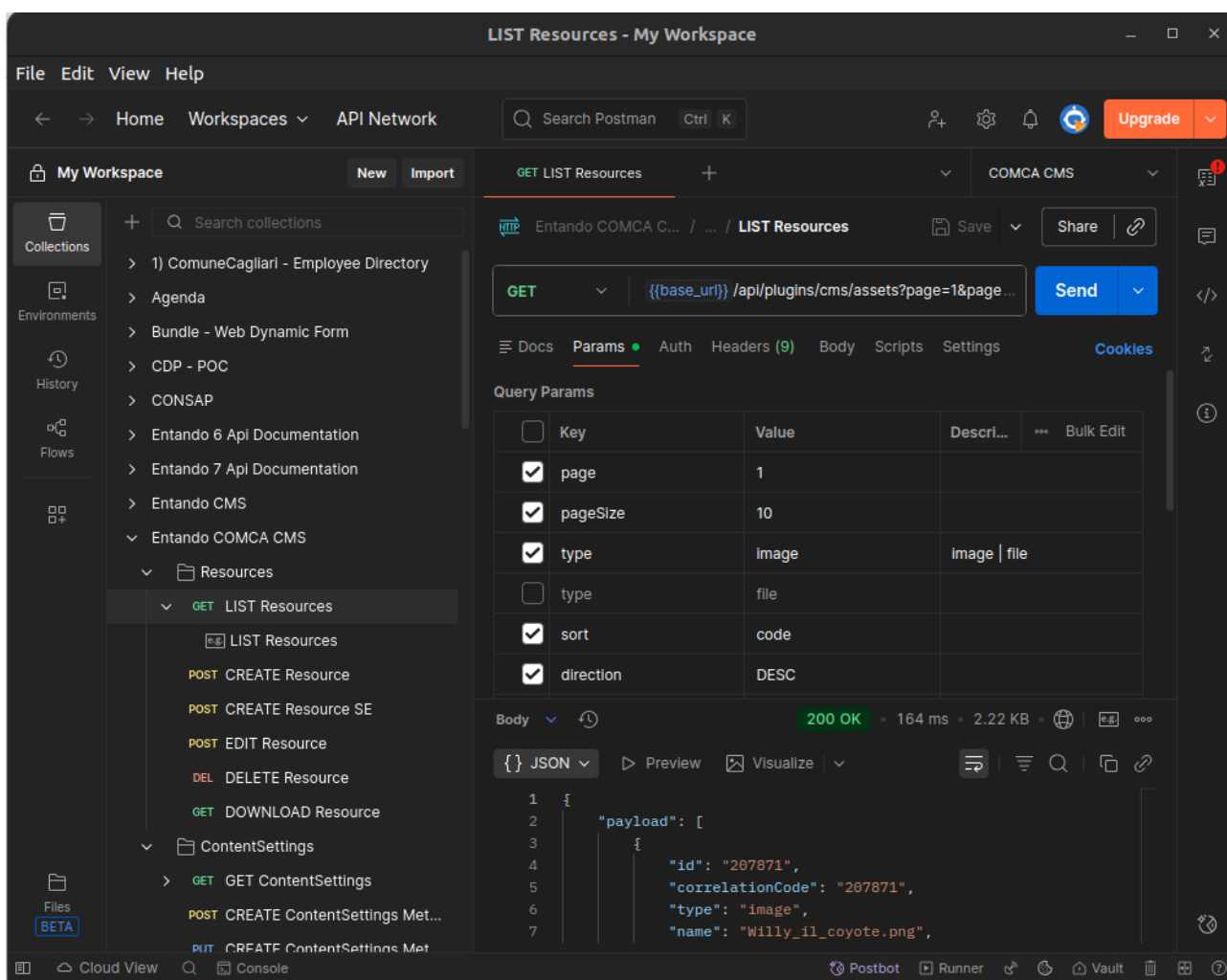
Da questo momento, tutti gli endpoint della collection ereditano il token negoziato in questa fase. Alla scadenza del token, Postman offre la possibilità di rinegoziazione tramite refresh token in maniera immediata.

In caso di configurazione errata, Postman segnalerà un errore; solitamente si tratta di:

- **kc_base_url** errato
- **base_url** di callback errato (non configurato correttamente sul client)
- *realm* errato
- *client id / secret* non corretti

Esempio di invocazione e note generali

Ottenuto l'access token, scegliere una qualsiasi REST dalla collection, ad esempio "List resources" e premere "Send". Si dovrebbe avere un risultato simile alla schermata sottostante



Questa invocazione include alcuni parametri di ricerca passati tramite query string. In generale, quando possibile, viene sempre mostrato come configurare i filtri di ricerca. Nel caso dell'endpoint dedicato alla lista dei contenuti, l'esempio relativo ai filtri è suddiviso in più chiamate, tutte riferite allo stesso endpoint. Questa scelta è stata fatta per rendere la documentazione più leggibile.

Tutti i filtri illustrati possono comunque essere combinati liberamente tra loro, purché vengano utilizzati sull'endpoint corretto.

entando

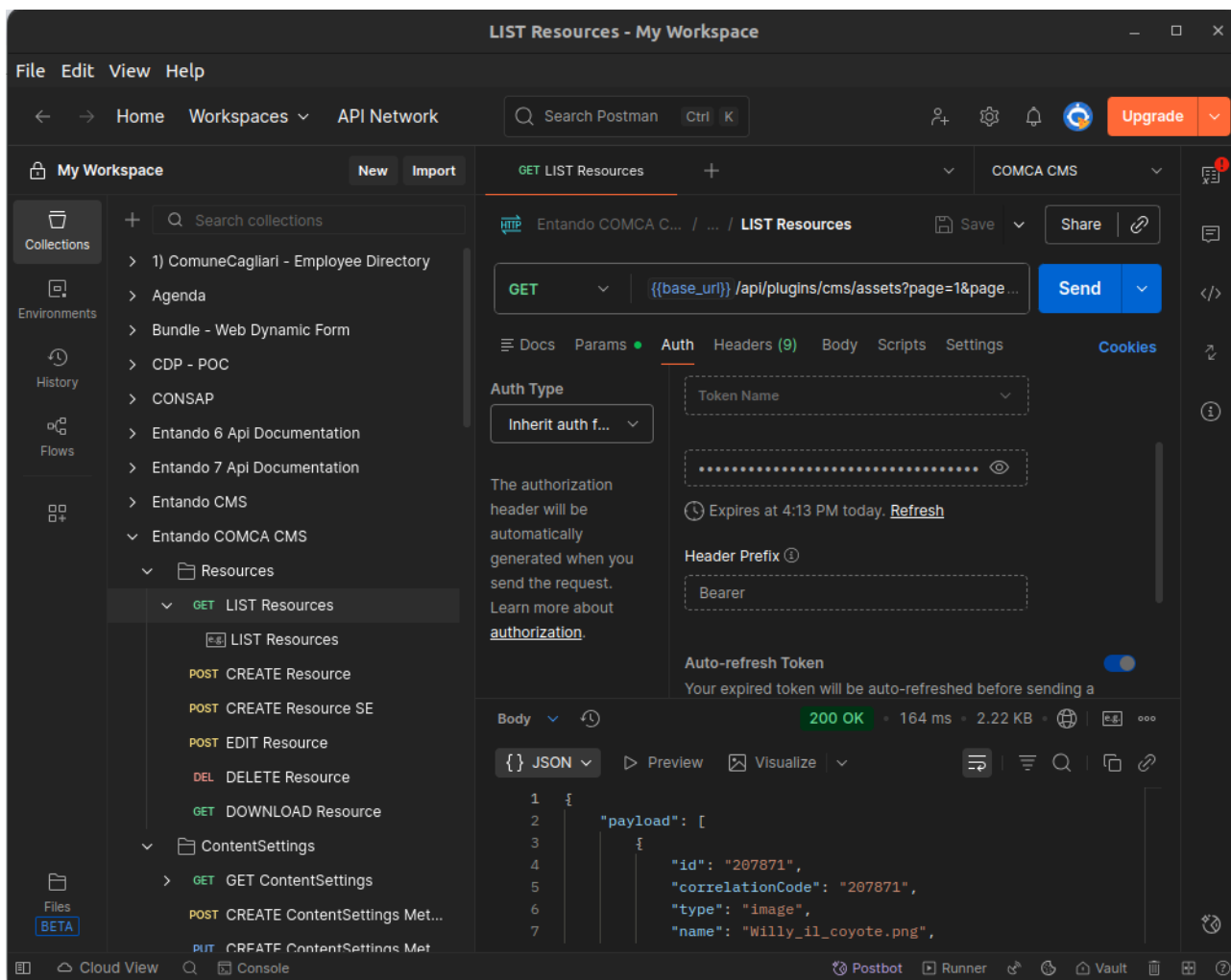
Oggetti diversi utilizzano gli stessi filtri assegnando valori a proprietà differenti. Di conseguenza, un filtro valido per la lista dei contenuti non può essere utilizzato in modo intercambiabile con quello per la lista delle pagine, anche se a livello concettuale e dichiarativo risultano molto simili.

Alcune proprietà possono assumere valori predefiniti: ad esempio, la proprietà *type* può essere valorizzata come *image* o *file* a seconda del tipo di ricerca. Le opzioni disponibili sono generalmente indicate nella descrizione del parametro oppure, in alcuni casi, lo stesso parametro può essere presente in forma disabilitata con il valore alternativo, così da mostrare chiaramente tutte le possibilità.

Alcune property, ad esempio quelle della paginazione, sono invece disponibili per tutti gli endpoint che producono liste.

Refresh dell'access token

Alla scadenza, l'access token può essere rinegoziato. La sua validità, in termini di durata temporale è un parametro che si può impostare nella configurazione di Keycloak.



Fare clic su “Refresh” per ottenere un nuovo access token.

Qualora non sia possibile rinegoziare il token bisognerà ripetere la [procedura di autenticazione](#) da capo.

CONTENT CMS

L'endpoint per i contenuti del cms è il seguente:

None

```
{{base_url}}/api/plugins/cms/contents
```

La ricerca per i contenuti avviene tramite filtri e parametri in query string. La collection contiene numerosi esempi di ricerca, per metadati e per attributo singolo.

Filtri per attributo

I filtri per attributo del CMS hanno questa forma:

None

```
filters[0].entityAttr:data_iniz  
filters[0].type:date  
filters[0].operator:gt  
filters[0].value:2005-02-13 01:00:00
```

Questo filtro, per esempio, ricerca i contenuti il cui attributo *data_iniz* ha un valore maggiore di quello dato nel campo value.

I possibili valori per la property type sono:

- **date** usato per la ricerca in attributi di tipo data
- **string** usato per gli attributi di tipo testo
- **boolean** usato per gli attributi di tipo booleano
- **number** usato per gli attributi di tipo numerico

Filtro per metadati

Un filtro per metadati ha questa struttura:

None

```
filters[0].operator:eq  
filters[0].attribute:typeCode  
filters[0].value:EVN
```

Questo filtro, per esempio, restringe la ricerca a tutti i tipi di contenuto di tipo EVN.

NOTA: il sistema distingue tra filtri per attributo e per metadati in base alla presenza della *property* *attribute*.

I possibili valori per la *property attribute* sono:

- **lastmodified** seleziona la data di ultima modifica
- **typeCode** seleziona per tipo di contenuto
- **description** seleziona per descrizione
- **id** seleziona per ID del contenuto

Concatenazione dei filtri

È sempre possibile aggiungere e combinare filtri multipli in sequenza: la numerazione parte sempre da zero e non sono ammessi “vuoti” nella numerazione: es. filters[0], filters[2] e filters[3] non sono una sequenza valida.

Le operazioni ammesse, ovverosia i valori della *property operator*, sono:

- **gt** greater than (maggiore di)
- **lt** lower than (minore di)
- **eq** equals (uguale a)
- **not** (diverso da)
- **like** ricerca per parte di testo (che contiene la stringa indicata)

Per restringere la ricerca ad un attributo per intervallo è necessario creare due filtri, uno per il limite superiore e uno per quello inferiore: c'è un esempio apposito nella collection “LIST content by attribute (date interval)”.

Stato pubblicazione

Entando per impostazione predefinita, cerca fra i contenuti in stato *DRAFT*. Per restringere la ricerca ai soli contenuti pubblicati si utilizzi la property *status* che può assumere i valori:

- **draft** per ricercare fra i contenuti non pubblicati
- **published** per i contenuti approvati per la pubblicazione

Escaping

A seconda del servlet container che esegue la de-app (es. Tomcat 9), è necessario fare l'escape anche delle parentesi quadre dei filtri:

None

```
filters%5B0%5D.entityAttr:hypertext  
filters%5B0%5D.type:string  
filters%5B0%5D.operator:like  
filters%5B0%5D.value:Italiano
```